PlayBetterGG -
tu mejor relación con los videojuegos

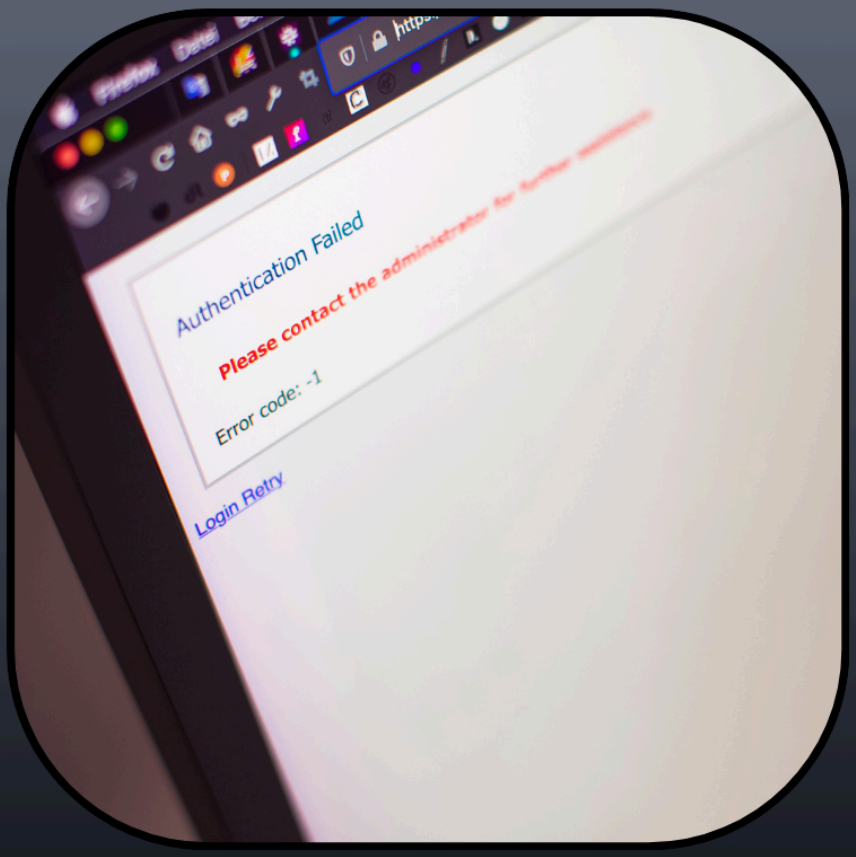
Esquema explicativo

Autor: César García Periáñez

2º CFGS Administración de Sistemas Informáticos en la Red

IES Albarregas

Fecha de entrega: 23-03-2026



Índice

Índice 2
Empresa: PlayBetterGG 3
Vista general del proyecto 3
Vista general del proyecto (esquema visual) 4
Infraestructura empresarial interna 5
Router 5
Switches 5
Windows Server - Active Directory 5
Estructura organizativa 6
Grupos globales 6
Grupos dentro de unidades organizativas anidadas: 6
Windows Server - Otros servicios integrados 7
Aplicación Web 8
Stack tecnológico 8
Integración del repositorio en GitHub 9
Esquema de red de aplicación web 9
Funcionalidades principales de la aplicación 10
Gestión de usuarios y roles 10
Sistema de gestión de incidencias (Tickets) 11
Funcionalidades 11
Flujo de trabajo 11
Gestión de citas y sesiones 12
Gestión de datos 12
Monitorización del servidor desde la aplicación 14
Archivos implicados 14
Información monitorizada 14
Conclusión general 15
Bibliografía 16
Anexo I: Requisitos mínimos y cómo se cumplen 17
Administración de sistemas operativos 17
Python 18
Seguridad y alta disponibilidad 18
Diseño, gestión y administración de la base de datos 19
Implementación de aplicaciones web 20

Esquema del Proyecto Intermodular – ASIR2

Empresa: PlayBetterGG

Vista general del proyecto

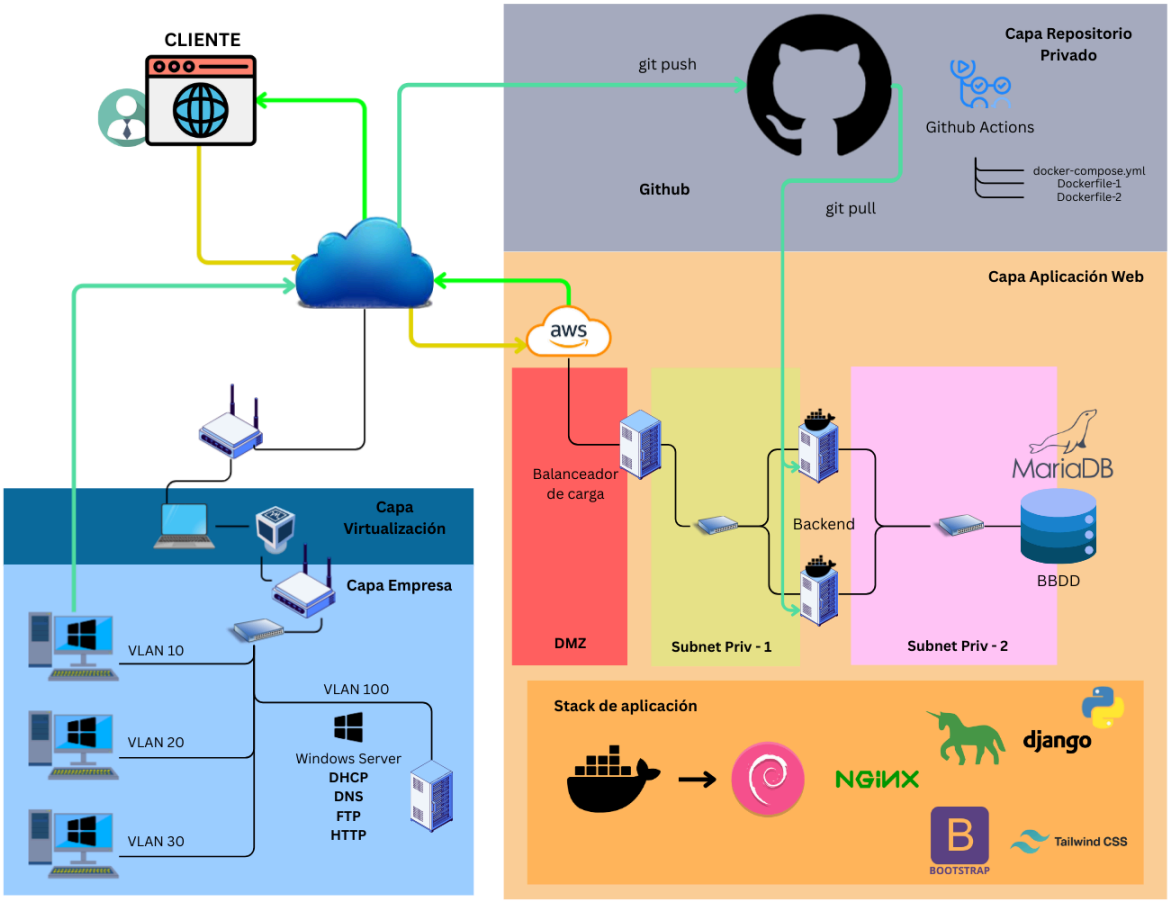
PlayBetterGG es una empresa ficticia orientada a:

- Apoyar a personas con adicciones comportamentales, especializándose en la adicción a los videojuegos, mediante una aplicación web propia para gestión de clientes y servicios.

El proyecto se divide en **dos grandes bloques**:

1. Infraestructura empresarial (Red interna, Windows Server,).
2. Desarrollo de aplicación web (Linux + Django + Base de datos).

Vista general del proyecto (esquema visual)



Infraestructura empresarial interna

La infraestructura interna está compuesta de cuatro piezas principales:

- Router
- Switches
- Equipos clientes para los trabajadores de la empresa
- Impresoras para los trabajadores de la empresa
- Servidor Windows 2022, que ejerce de nodo principal. Gestiona:
    - DHCP
    - DNS
    - Active Directory

Router

Se simula mediante un equipo Ubuntu Server con reenvío de puertos y reglas iptables.

Switches

Los switches de la red se simulan mediante redes internas de VirtualBox, que permiten la segmentación lógica del tráfico entre máquinas virtuales.

Windows Server - Active Directory

Se implementa un dominio corporativo con:

- Una **OU por cada equipo o unidad organizativa**.
- Creación de:

    - Usuarios
    - Grupos
    - Asociación a equipos físicos
- Control estricto de permisos para proteger datos sensibles.

Estructura organizativa

Grupos globales

- Directivos
- Admins
- JefeArea_Global
- Trabajadores_Global
- Becarios_Global

Grupos dentro de unidades organizativas anidadas:

OU - Producto Tecnológico

Grupos:

- Desarrollo Web
- Becarios_DWEB

OU - Desarrollo Personal

Grupos:

- Psicólogos
- Becarios_PSICO

OU - Márketing y RRSS

Grupos:

- Márketing
- Becarios_MARK

OU - Contable y Finanzas

- Contable
- Becarios_FINA

OU - Gestores, administrativos y RRHH

- Administrativo
- Becarios_GEST

Windows Server - Otros servicios integrados

- Servicio DHCP
- Servicio DNS
- Servicio de impresoras

Aplicación Web

La empresa se especializa en terapias a pacientes con problemas de adicción comportamentales. Para ello utiliza una aplicación web desplegada en un proveedor de infraestructura en nube, accesible desde internet para cualquier usuario promedio.

Stack tecnológico

El stack tecnológico de la aplicación web comprende:

- Instancia EC2 pública, Apache con Certbot como proxy y balanceo de carga
- Dos o más instancias con contenedores (docker-compose):
    - Debian con **Apache**
    - Debian con **Gunicorn/Django (Modelo-Vista-Controlador)**
- Instancia EC2 con base de datos **MariaDB**
- Instancia EC2 con memoria caché **Redis** para gestión de sesiones

Esta aplicación web dispone de un portal de login para los clientes registrados. Para los usuarios de la red interna empresarial, logueados mediante el protocolo LDAP de Active Directory, se establece un logueo LDAP automático. Se detalla este procedimiento en el apartado [[LDAP]]

Para usuarios externos, Django, Redis y MariaDB gestionan las credenciales de forma típica, comprobando el inicio de sesión contra la base de datos y manteniendo al usuario conectado gracias la memoria caché.

Integración del repositorio en GitHub

El código fuente del proyecto está disponible en el siguiente repositorio:

https://github.com/cgarciap58/cesar_garcia_proyecto_intermodular

La instancia EC2 en AWS se actualiza automáticamente mediante un script de automatización Bash

Esquema de red de aplicación web

Funcionalidades principales de la aplicación

Gestión de usuarios y roles

Roles definidos:

- Cliente (paciente)
- Psicólogo/a
- Jefe de desarrollo
- Desarrollador

Cada rol tiene:

- Vistas específicas
- Permisos definidos en Python (Django)
- Acceso controlado según su función en la empresa

Sistema de gestión de incidencias (Tickets)

Funcionalidades

- Registro de errores
- Clasificación por tipo
- Asignación a desarrolladores
- Estados posibles:
    - Resuelta
    - No reproducible
    - No considerada incidencia real

Flujo de trabajo

1. Persona reporta incidencia.
2. Jefe de desarrollo revisa.
3. Asigna a desarrollador.
4. Desarrollador actualiza estado.
5. Se cierra o descarta la incidencia.

Gestión de citas y sesiones

- Solicitud de citas (cliente).
- Gestión de agenda (psicólogo).
- Creación de notas internas por sesión.
- Almacenamiento en base de datos mediante ORM de Django.

Gestión de datos

La aplicación utiliza:

- Base de datos relacional MariaDB.
- ORM de Django para acceso seguro.
- Modelos principales:
    - Datos personales
    - Datos laborales
    - Datos de clientes
    - Credenciales de acceso
    - Gestión de citas
    - Seguimiento profesional
    - Sistema de recomendaciones

Internamente se trabaja con:

- Listas de incidencias
- Listas de usuarios

- Estados predefinidos

Monitorización del servidor desde la aplicación

Se implementa un panel de administración que:

- Ejecuta comandos Bash desde Python.
- Usa la librería `subprocess`.
- Muestra resultados en el navegador.

Archivos implicados

- `views.py` → ejecución de comandos
- `models.py` → modelo Ticket
- `urls.py` → rutas
- `templates/tickets_dashboard.html` → vista
- `settings.py` → configuración

Información monitorizada

- Sistema (uname, uptime, hostname)
- CPU (lscpu, top, mpstat)
- Memoria (free, vmstat)
- Disco (df, lsblk)
- Procesos (ps aux)
- Red (ip a, ss)

Conclusión general

El proyecto integra:

- Administración de sistemas (AD, servicios de red).
- Seguridad y segmentación por roles.
- Desarrollo backend en Python con Django.
- Persistencia de datos mediante base de datos relacional.
- Sistema realista de tickets.
- Monitorización del servidor desde la propia aplicación web.

Es un proyecto completo, escalable y alineado con los objetivos de ASIR2, combinando infraestructura, servicios de red y desarrollo de aplicaciones.

Si quieres, puedo prepararte una versión todavía más resumida tipo “esquema para exponer en 5 minutos” o una versión más técnica para memoria final.

Bibliografía

- https://realpython.com/django-nginx-gunicorn/
- https://www.youtube.com/watch?v=WtdefC1WJgE
- https://mariadb.org/en/
- https://www.w3schools.com/django/django_intro.php
-

Anexo I: Requisitos mínimos y cómo se cumplen

En este anexo se detallan los requisitos mínimos de cada asignatura de implementación obligatoria en el proyecto y cómo están cubiertas en mi proyecto, de forma clara y resumida.

Administración de sistemas operativos

1. Uno o varios scripts para la administración de servidor Linux
    a. Dockerfile que ejecuta varios comandos Bash para montar el contenedor Django
    b. Script BASH de migraciones de la BBDD mediante llamada a manage.py, con output a la CLI para supervisar el sistema
2. Dominio de empresa en Windows Server
    a. Detalladas las OUs, grupos, usuarios y GPOs
    b. Detallados los servicios implementados
3. Parte Linux opcional. Instalación de servidor LDAP e infraestructura del mismo
    a. No se implementa OpenLDAP, dado que ya se dispone de gestión del dominio mediante Active Directory en Windows Server.
    b. Sí se usan los protocolos LDAP y Kerberos con los equipos Linux encargados de la aplicación web, para hacer el SSO posible.
4. Parte Windows opcional - Desarrollo de script Powershell
    a. Se dispone de un script ps1 encargado de gestionar la creación de usuarios, grupos y OUs de forma sencilla mediante CLI, permitiendo añadir usuarios, grupos y OUs de forma anidada, de uno en uno o mediante carga de lista csv

Python

1. Aplicación en Python
    a. La aplicación web desplegada se despliega en su totalidad utilizando python en el backend. Todos el código de control está escrito en python puro, expandiendo sobre las clases y métodos aportados por el framework Django.
2. Gestión de incidencias mediante entorno gráfico
    a. La propia aplicación facilita gestión de incidencias dentro de la propia página web, utilizando la propia gestión de usuarios y autenticación ya existente en la BBDD que utiliza Django, que a su vez está integrada y provisionada con el Active Directory

Seguridad y alta disponibilidad

1. Definición e implementación de políticas de seguridad
    a. Se aplican políticas básicas de seguridad, como cambio de contraseñas obligatorios a los usuarios, contraseñas fuertes, intentos máximos de log-ins, etcétera.
2. Análisis de vulnerabilidades
    a. Se realiza un primer análisis de vulnerabilidades, definiendo riesgos e impactos
    b. Se menciona la posibilidad de contratar una auditoría externa (pentesting) para detectar vulnerabilidades en nuestra red interna y en nuestra aplicación web
3. Aplicación de reglas ip tables
    a. El router de la red interna deniega todo el tráfico por defecto, permitiendo después tráfico al exterior de la red, y entre equipos de la red interna y el servidor.

b. Se aplican reglas de firewall mediante políticas de seguridad y compartimentalización de redes en AWS

Diseño, gestión y administración de la base de datos

1. Diseño lógico
    a. Se diseña un modelo ER para el esquema de la base de datos.
    b. Se verifica la normalización del modelo
    c. Se esquematiza visualmente y se presenta
2. Diseño físico
    a. Se implementa el modelo ER mediante las clases “Modelo” en Django (Models, dentro de esquema Modelo-Vista-Controlador)
    b. Se procede a la migración de los objetos-(de la clase “modelos”) de Django a objetos (tablas) dentro de una BBDD MariaDB alojada en un servidor distinto (Debian) dentro de la red.
3. Reglas específicas
    a. Se corta el acceso a la BBDD desde el exterior mediante reglas de enrutamiento en AWS, salvo que sea desde las IPs asociadas a las aplicaciones Django. Sólo puerto 3306.
    b. Se duplica la misma configuración dentro de MariaDB. Sólo puerto 3306 desde las IPs relevantes.
    c. Se permite acceso SSH desde la IP bastión de AWS.
    d. Se loguean intentos fallidos de conexión

Implementación de aplicaciones web

1. Infraestructura en general del despliegue en la web
    a. Se despliega en AWS un balanceador, al menos dos servidores de backend, un servidor de memoria caché, y un servidor de base de datos. Se dividen en redes separadas y se asegura que la base de datos no está expuesta a la red pública. Se tiene DMZ.
2. Servicios de la intranet simulada (Vagrant/VirtualBox)
    a. Los servidores DHCP, DNS o FTP, entre otros servicios básicos internos, se virtualizan mediante Vagrant/VirtualBox en una red interna.
3. Máquina Debian sin entorno gráfico
    a. El router se monta en un Debian mediante CLI
4. Mejora 1 - Firewall
    a. Ip tables en el router
5. Mejora 2 - DMZ
    a. Tan sólo el balanceador está expuesto al tráfico del internet, y sólo en los puertos 80 y 443
6. Mejora 3 - Balanceo en la base de datos
    a. No implementado
7. Mejora 4 - Sistema de archivos compartidos para el backend (NFS o similar)
    a. Se trabaja con Github Actions y crontab para que se desplieguen automáticamente los cambios realizados a la rama 'main', lo cual crea una 'única fuente de verdad' en el despliegue de la aplicación.

Servicios en red

1. Servicio DHCP
    a. Asignación automática de IPs en la red interna
        i. Las IPs 192.168.1.51-192.168.1.200 están libres para asignación dinámica
    b. Reservas para servidores estáticos y equipos críticos
        i. 192.168.1.1 - Router Debian - Puerta de enlace
        ii. 192.168.1.2 - Windows Server - DHCP, DNS y AD
        iii. 192.168.1.3 - Servidor FTP
    c. Opcional - Definición de diferentes rangos de LAN
        i. 192.168.1.11 - 192.168.1.30 - IPs para gestión remota switches
        ii. 192.168.1.31 - 192.168.1.50 - Impresoras

2. Servicio DNS
    a. Implementación de dominio interno
        i. Playbetter.gg
    b. Resoluciones directas e inversas de los nombres de equipo
        i. Se establecen las siguientes:
            1. mainServer.playbetter.gg - 192.168.1.2
            2. ftpServer.playbetter.gg - 192.168.1.3
    c. Integración con el servidor web y el servidor FTP
        i. Se resuelven los dominios de dichos servidores

3. Servicio FTP
    a. Creación de un servidor FTP interno para el intercambio de archivos entre departamentos
        i. Carpeta ‘ANUNCIOS PÚBLICOS’ visible a todos los departamentos
    b. Implementación de usuarios con diferentes permisos y directorios personales o compartidos
        i. Una carpeta por cada departamento, automatización de login FTP mediante AD
    c. Configuración de acceso FTP seguro

4. Servicio HTTP
    a. Despliegue de una página web corporativa en un servidor
        i. Se despliega página web interna que muestra la carga del servidor principal en un dashboard
        ii. Se despliega la aplicación web principal en AWS
    b. Acceso interno mediante el dominio gestionado por el DNS
        i. El DNS redirecciona a la ip pública de la aplicación web, dando redundancia al servicio de registro DNS externo (https://my.noip.com)